



ISLAMIC UNIVERSITY OF TECHNOLOGY

EEE 4710

Machine Learning and Artificial Intelligence

Project Title:

**An Explainable and Mobile-Deployable Deep Learning Framework for
Diabetic Foot Ulcer Detection Using Smartphone Images**

Group Name: Algorithm Architects

Team Member	Student ID
Ar-Rafi Ishraq	210021330
Mohd. Abdullah Abrar	210021206
Md. Sabbir Hossain Tashrif	210021234

1. Problem Definition

Diabetic Foot Ulcer (DFU) is a major complication of diabetes and a leading cause of lower-limb amputation worldwide. Early detection through visual inspection can significantly reduce severe outcomes. With the increasing availability of smartphones, mobile-based remote DFU screening systems offer a cost-effective solution, especially in low-resource and rural settings.

Existing deep learning approaches for DFU detection primarily focus on binary classification using clinical images. However, most of these systems operate as black boxes, lack explainability, and do not address real-world challenges such as variable image quality, uncertainty in predictions, and suitability for mobile deployment.

This project aims to develop a machine learning-based image classification system that can automatically detect whether a foot image contains an ulcer or not. Deep learning, particularly Convolutional Neural Networks (CNNs), can effectively analyze visual patterns in medical images, making them suitable for this task.

Most current DFU detection studies using public datasets focus only on classification accuracy and do not:

- Provide visual explanations to support clinical trust
- Assess model confidence or uncertainty
- Consider real-world mobile image variability
- Evaluate suitability for lightweight, on-device deployment

This limits their applicability in real telemedicine scenarios.

The objectives of this project are:

1. To develop a deep learning model for automatic DFU detection using smartphone images.
2. To integrate explainable AI (XAI) techniques to visually highlight ulcer-relevant regions.
3. To estimate prediction confidence/uncertainty to improve clinical reliability.
4. To optimize the model for mobile deployment with low computational cost.
5. To evaluate model robustness under real-world imaging conditions

To evaluate model robustness under real-world imaging conditions.

2. Dataset Preparation

In this project we are using the publicly available Kaggle dataset:

Diabetic Foot Ulcer (DFU) Dataset:

<https://www.kaggle.com/laithjj/diabetic-foot-ulcer-dfu>

Dataset Characteristics:

- Contains images of healthy feet and diabetic foot ulcers
- Captured using smartphone devices
- Includes variations in lighting, background, and angle
- Suitable for real-world telemedicine research
- Binary labels: Normal vs Ulcer

Data Preprocessing:

Steps performed:

- Removed duplicate and unclear images
- Eliminated confusing samples (e.g., callus, mixed images)
- Organized dataset into train, validation, and test sets
- Balanced the test dataset for fair evaluation
- Resized all images to a fixed dimension (224x224)

Final structure:

```
train/  
  nonulcer/  
  ulcer/  
val/  
  nonulcer/  
  ulcer/  
test/  
  nonulcer/  
  ulcer/
```

Name	Date modified	Type	Size
test	4/19/2026 2:48 AM	File folder	
train	4/19/2026 2:47 AM	File folder	
val	4/19/2026 2:47 AM	File folder	

Name	Date modified	Type	Size
nonulcer	4/19/2026 2:48 AM	File folder	
ulcer	4/19/2026 2:51 AM	File folder	

3. Model Development

As we are dealing with images, we need to use CNN models. We will use two models MobileNetV2 and Efficientnet-B0 , then compare the performance and results and select the best one for Mobile App.

Steps:

- Loaded MobileNetV2 and Efficientnet-B0 with pretrained ImageNet weights
- Removed top classification layer (include_top=False)
- Added custom Dense layers:
 - Dense(128) + ReLU
 - Dropout(0.5)
 - Dense(1) + Sigmoid

Training:

- Used train/validation split
- Optimizer: Adam
- Loss: Binary Crossentropy
- Epochs: 10

(i) MobileNetV2:

```
# LOAD PRETRAINED MODEL
base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)
```

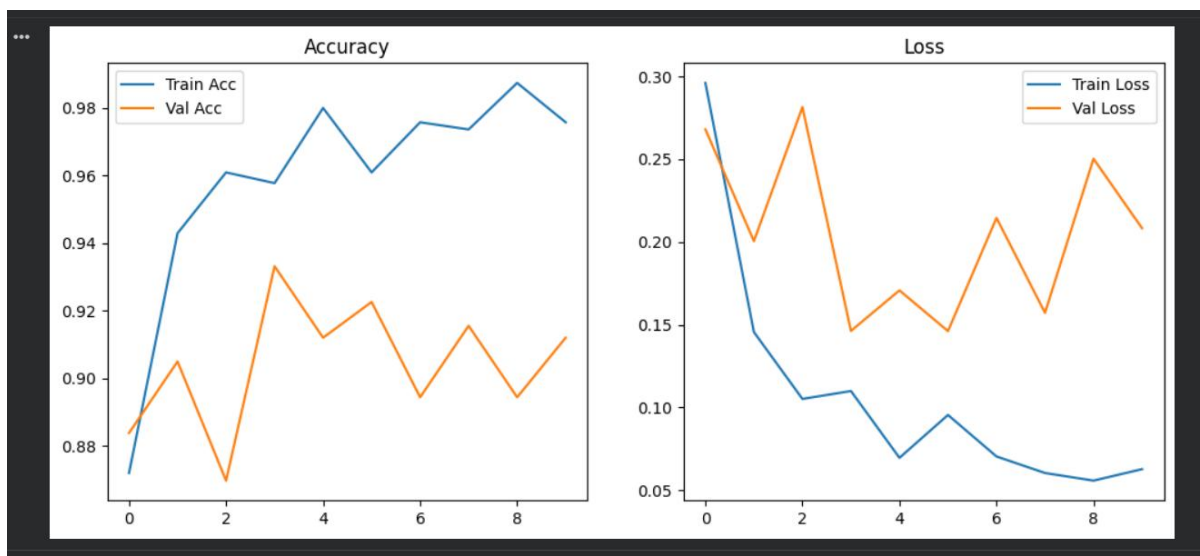
Compile Model

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

Model training:

```
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10
)
```

```
Epoch 1/10
30/30 ————— 464s 15s/step - accuracy: 0.8720 - loss: 0.2960 - val_accuracy: 0.8838 - val_loss: 0.2680
Epoch 2/10
30/30 ————— 16s 552ms/step - accuracy: 0.9429 - loss: 0.1456 - val_accuracy: 0.9049 - val_loss: 0.2005
Epoch 3/10
30/30 ————— 17s 567ms/step - accuracy: 0.9608 - loss: 0.1052 - val_accuracy: 0.8697 - val_loss: 0.2814
Epoch 4/10
30/30 ————— 17s 562ms/step - accuracy: 0.9577 - loss: 0.1099 - val_accuracy: 0.9331 - val_loss: 0.1462
Epoch 5/10
30/30 ————— 16s 550ms/step - accuracy: 0.9799 - loss: 0.0696 - val_accuracy: 0.9120 - val_loss: 0.1707
Epoch 6/10
30/30 ————— 17s 556ms/step - accuracy: 0.9608 - loss: 0.0955 - val_accuracy: 0.9225 - val_loss: 0.1461
Epoch 7/10
30/30 ————— 17s 562ms/step - accuracy: 0.9757 - loss: 0.0704 - val_accuracy: 0.8944 - val_loss: 0.2145
Epoch 8/10
30/30 ————— 16s 544ms/step - accuracy: 0.9735 - loss: 0.0604 - val_accuracy: 0.9155 - val_loss: 0.1572
Epoch 9/10
30/30 ————— 17s 560ms/step - accuracy: 0.9873 - loss: 0.0558 - val_accuracy: 0.8944 - val_loss: 0.2502
Epoch 10/10
30/30 ————— 16s 551ms/step - accuracy: 0.9757 - loss: 0.0628 - val_accuracy: 0.9120 - val_loss: 0.2083
```



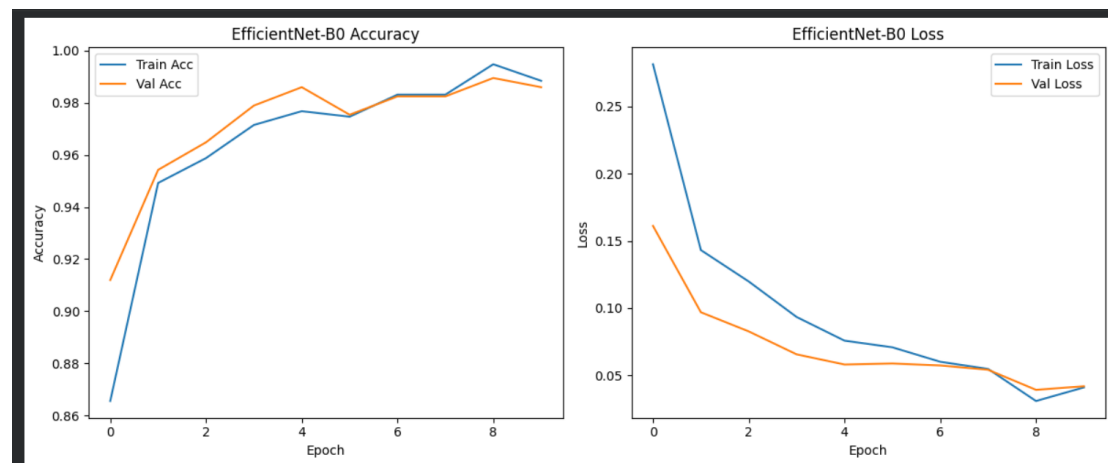
(ii) Efficientnet-B0 (2nd model):

Build EfficientNet-B0 model	Compile model
<pre>base_model_eff = EfficientNetB0(input_shape=(224, 224, 3), include_top=False, weights='imagenet') base_model_eff.trainable = False</pre>	<pre>model_eff.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])</pre>

Model Training:

```
history_eff = model_eff.fit(  
    train_generator_eff,  
    validation_data=val_generator_eff,  
    epochs=10  
)
```

... Epoch 1/10
30/30 ————— 77s 2s/step - accuracy: 0.8656 - loss: 0.2813 - val_accuracy: 0.9120 - val_loss: 0.1611
Epoch 2/10
30/30 ————— 17s 565ms/step - accuracy: 0.9492 - loss: 0.1432 - val_accuracy: 0.9542 - val_loss: 0.0970
Epoch 3/10
30/30 ————— 16s 547ms/step - accuracy: 0.9587 - loss: 0.1198 - val_accuracy: 0.9648 - val_loss: 0.0827
Epoch 4/10
30/30 ————— 17s 555ms/step - accuracy: 0.9714 - loss: 0.0935 - val_accuracy: 0.9789 - val_loss: 0.0657
Epoch 5/10
30/30 ————— 16s 550ms/step - accuracy: 0.9767 - loss: 0.0759 - val_accuracy: 0.9859 - val_loss: 0.0581
Epoch 6/10
30/30 ————— 17s 562ms/step - accuracy: 0.9746 - loss: 0.0709 - val_accuracy: 0.9754 - val_loss: 0.0589
Epoch 7/10
30/30 ————— 17s 565ms/step - accuracy: 0.9831 - loss: 0.0602 - val_accuracy: 0.9824 - val_loss: 0.0574
Epoch 8/10
30/30 ————— 17s 554ms/step - accuracy: 0.9831 - loss: 0.0548 - val_accuracy: 0.9824 - val_loss: 0.0542
Epoch 9/10
30/30 ————— 16s 550ms/step - accuracy: 0.9947 - loss: 0.0310 - val_accuracy: 0.9894 - val_loss: 0.0393
Epoch 10/10
30/30 ————— 16s 550ms/step - accuracy: 0.9884 - loss: 0.0411 - val_accuracy: 0.9859 - val_loss: 0.0420



4. Performance Evaluation:

(i) MobilenetV2:

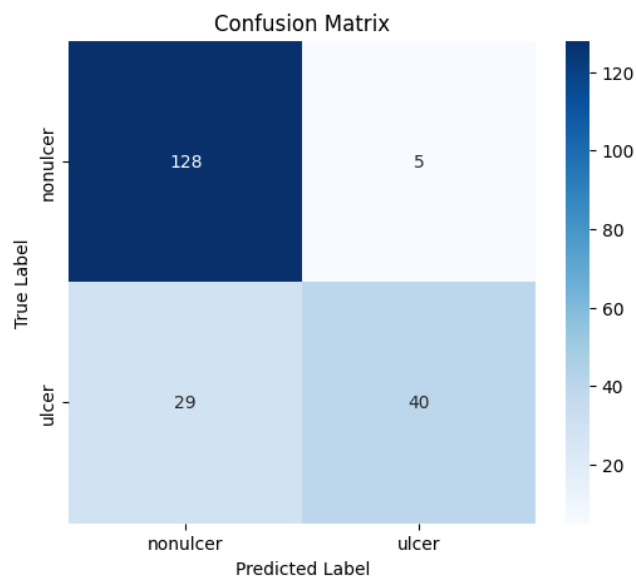
1. Accuracy:

```
Evaluate on Test Set

test_loss, test_acc = model.evaluate(test_generator)
print("Test Accuracy:", test_acc)

... 7/7 ————— 107s 18s/step - accuracy: 0.8317 - loss: 1.2410
Test Accuracy: 0.8316831588745117
```

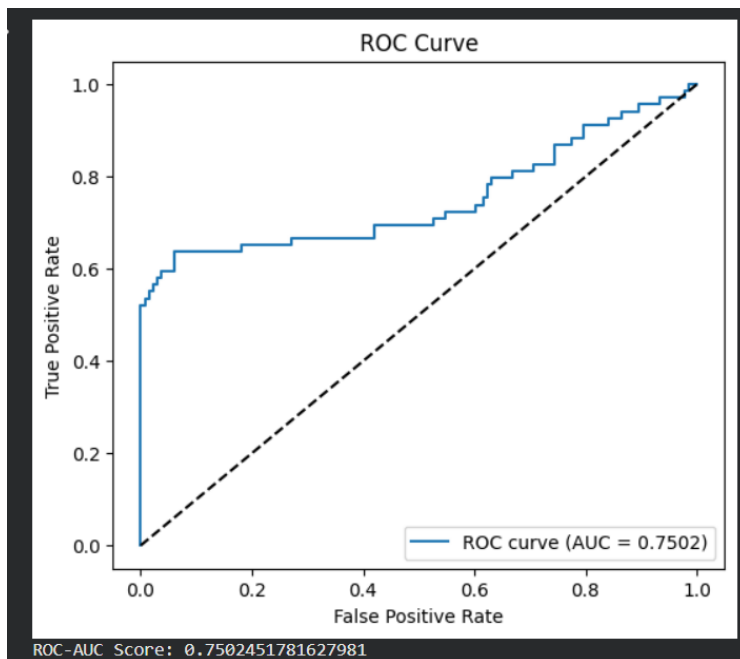
2. Confusion Matrix:



3. Other Performances:

```
... Classification Report:
```

	precision	recall	f1-score	support
nonulcer	0.82	0.96	0.88	133
ulcer	0.89	0.58	0.70	69
accuracy			0.83	202
macro avg	0.85	0.77	0.79	202
weighted avg	0.84	0.83	0.82	202



(ii) EfficientNet-B0:

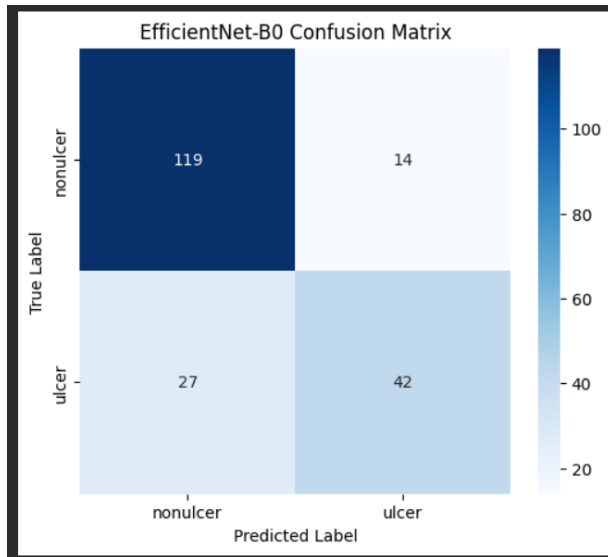
1. Accuracy:

Evaluate on test set

```
test_loss_eff, test_acc_eff = model_eff.evaluate(test_generator_eff)
print("EfficientNet-B0 Test Loss:", test_loss_eff)
print("EfficientNet-B0 Test Accuracy:", test_acc_eff)
```

```
7/7 1s 171ms/step - accuracy: 0.7970 - loss: 1.0220
EfficientNet-B0 Test Loss: 1.0219721794128418
EfficientNet-B0 Test Accuracy: 0.7970296740531921
```

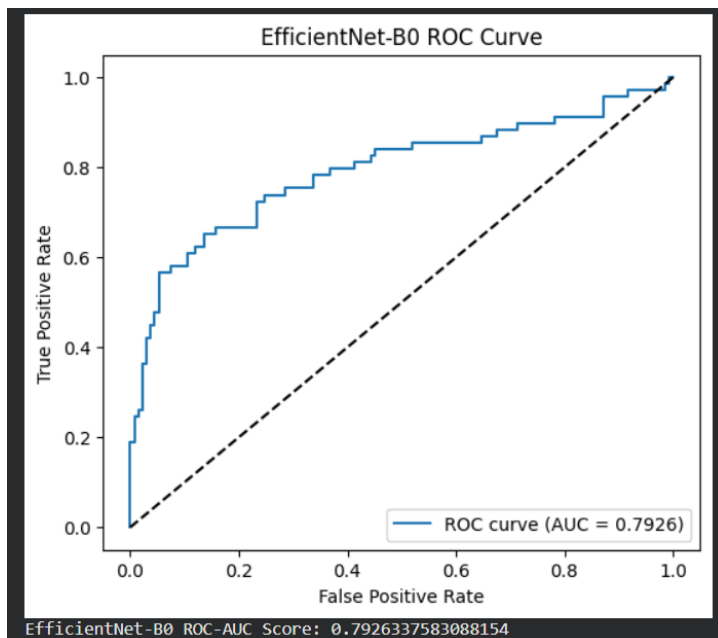

2. Confusion Matrix:



3. Other Performances:

... EfficientNet-B0 Classification Report:

	precision	recall	f1-score	support
nonulcer	0.82	0.89	0.85	133
ulcer	0.75	0.61	0.67	69
accuracy			0.80	202
macro avg	0.78	0.75	0.76	202
weighted avg	0.79	0.80	0.79	202



Comparison of these two models:

```
comparison_df
```

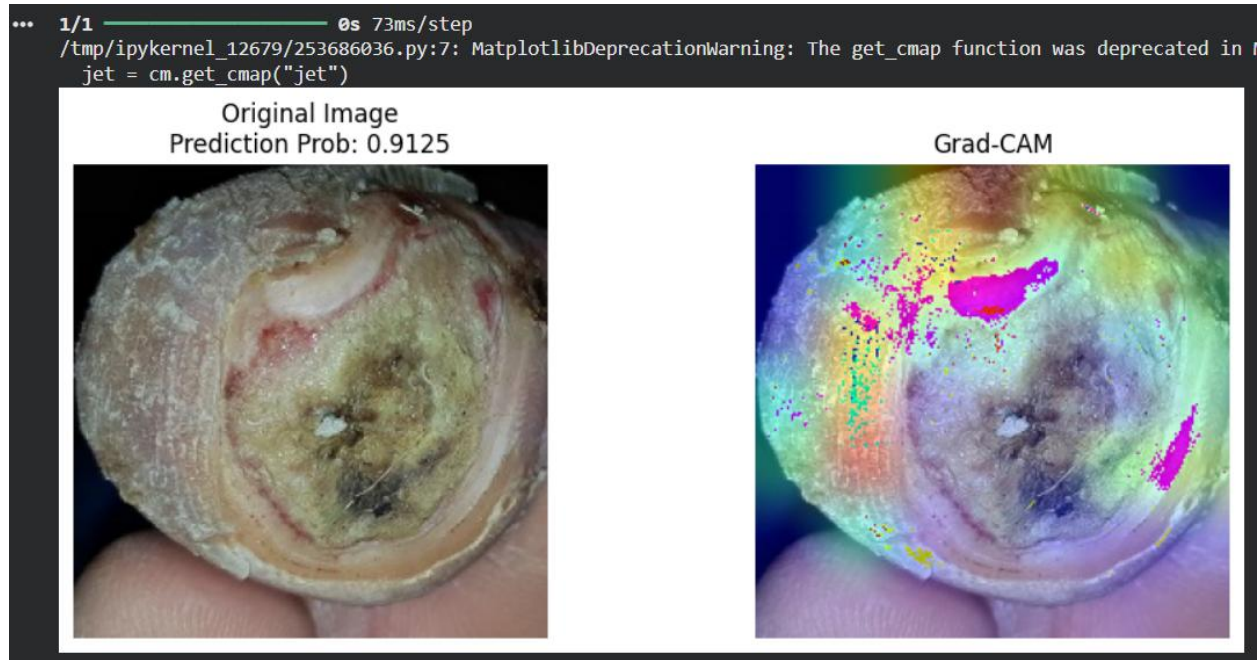
	Model	Test Accuracy	ROC-AUC	Model Size (MB)
0	MobileNetV2	0.8317	0.7502	11.01
1	EfficientNet-B0	0.7970	0.7926	18.03

Takeaway: MobileNetV2 is Best among these two.

Additional Implementations:

1. Grad-CAM:

This thing is done to know the reason behind the model prediction. It highlights the area of the feet for which it has detected true or false.



2. Monte Carlo Dropout for uncertainty estimation:

This is the strongest addition of our project.

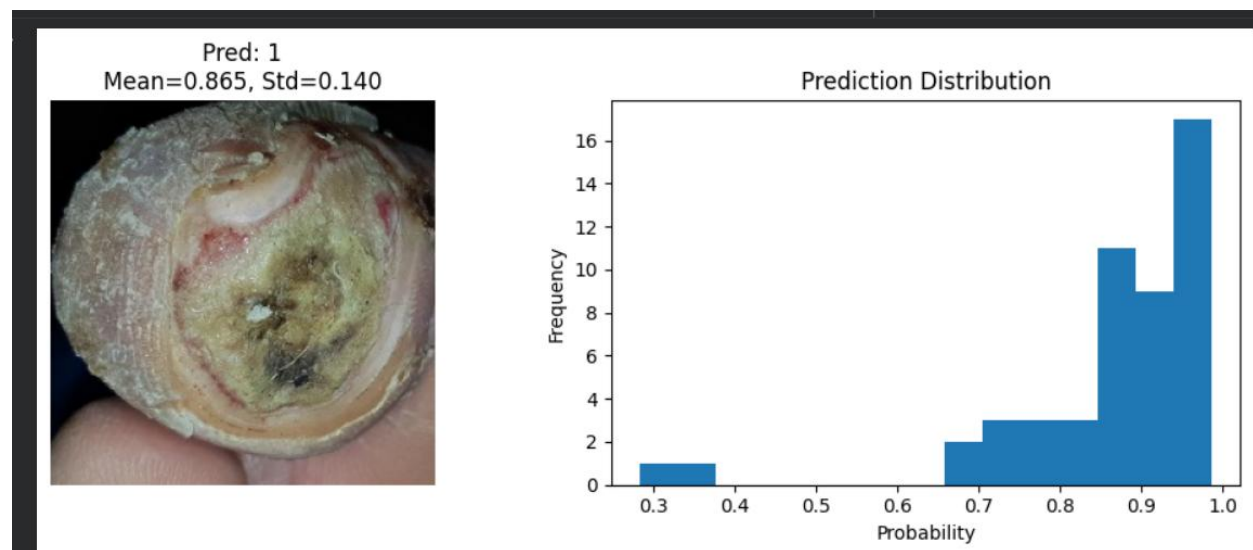
For one image, instead of predicting once, we predict multiple times with dropout active.

Then we compute:

- mean prediction
- standard deviation

Interpretation:

- high mean + low std → confident ulcer prediction
- low mean + low std → confident normal prediction
- mid mean or high std → uncertain case



Rules for confidence level:

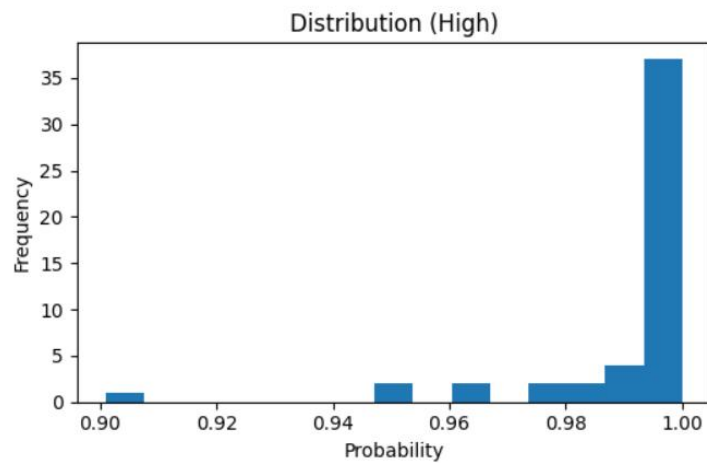
- High confidence:
mean > 0.9 OR mean < 0.1 AND std < 0.05
- Medium confidence:
mean moderately far from 0.5 AND std < 0.1
- Low confidence:
mean near 0.5 OR std high (> 0.1)

Define confidence function

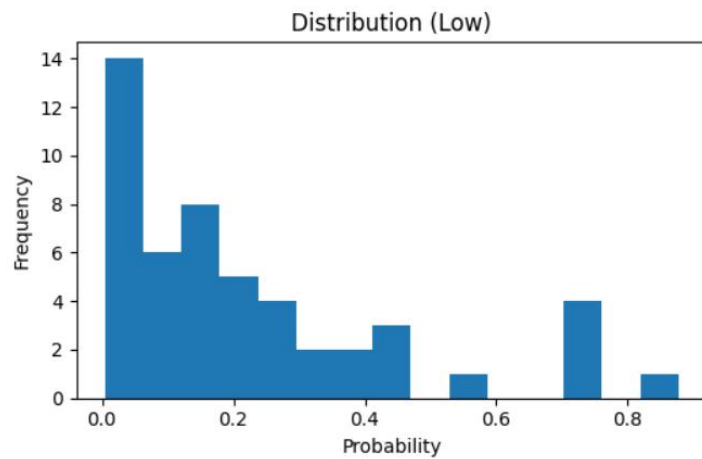
```
def get_confidence_level(mean, std):  
  
    # Strong confident prediction (very high or very low)  
    if (mean > 0.9 or mean < 0.1) and std < 0.05:  
        return "High"  
  
    # Moderately confident  
    elif (mean > 0.75 or mean < 0.25) and std < 0.1:  
        return "Medium"  
  
    # Otherwise uncertain  
    else:  
        return "Low"
```

Some Results:

Pred: 1
Mean=0.991, Std=0.017
Confidence: High



Pred: 0
Mean=0.227, Std=0.221
Confidence: Low



5. Deployment

The trained model was converted into TensorFlow Lite (TFLite) format for mobile deployment.

Steps:

- Converted model to .tflite
- Measured model size and inference time
- Built Android application using Android Studio

Generating TFLite for Mobile App

```
# CONVERT TO TFLITE

converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

tflite_model_path = '/content/drive/MyDrive/project4710/mobilenetv2_binary_v2.tflite'

with open(tflite_model_path, 'wb') as f:
    f.write(tflite_model)

print("TFLite model saved at:", tflite_model_path)

... Saved artifact at '/tmp/tmp9h_zygg1'. The following endpoints are available:
```

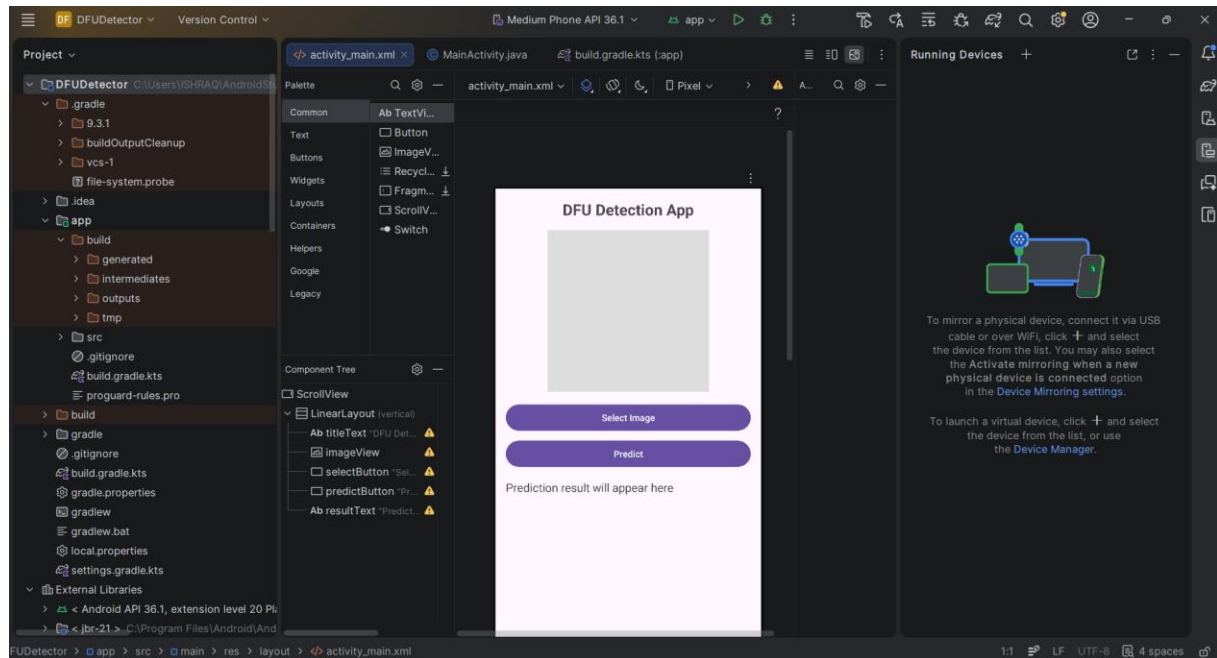
Comparison of TFLite with main MobileNetV2 model:

```
===== MODEL COMPARISON =====
Keras model size      : 11.01 MB
TFLite model size     : 9.07 MB
Keras inference time  : 110.24 ms
TFLite inference time : 13.77 ms
Keras prediction      : 0.997325
TFLite prediction     : 0.997325
```

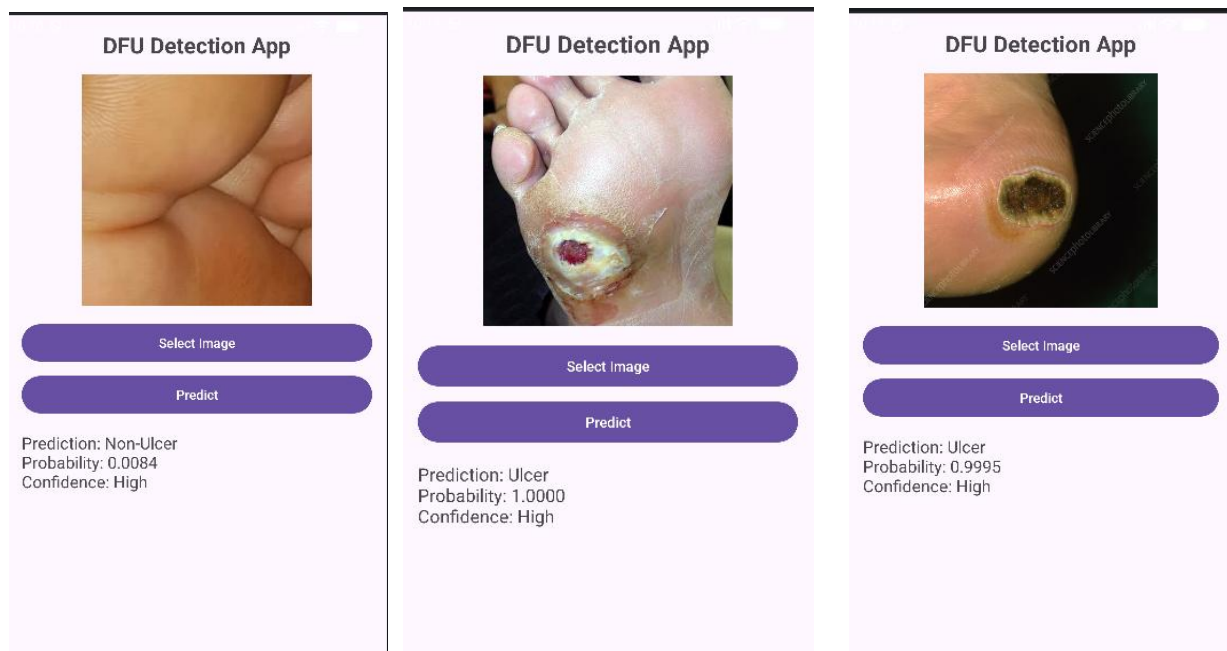
...			
cell output actions			
	Model	Size (MB)	Inference Time (ms)
	Keras (.h5)	11.01	110.24
1	TensorFlow Lite (.tflite)	9.07	13.77

Mobile App:

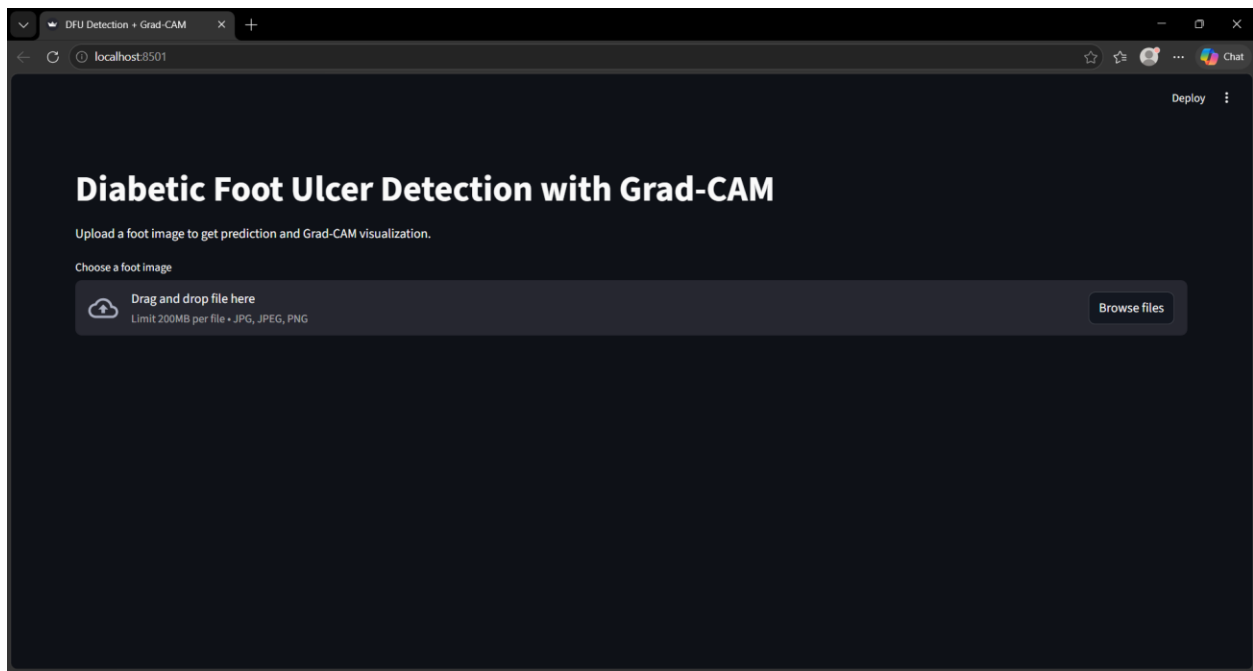
We developed the mobile app using Android Studio.



mobilenetv2_binary_v2.tflite model was used as base model to develop this app. This app was developed based on Android version 8, so that it can run on any mobile phone. The app is only 25MB, which is a very light weight app and suitable for any mobile. Below are some sample predictions by the app:



If someone face any issue while installing or using the app, for them we have made a simple Website where they can upload the image and find the result. In the web version Grad-CAM is also added, this thing was not added to the mobile app to make the app as light as possible.



Some predictions:

Diabetic Foot Ulcer Detection with Grad-CAM

Upload a foot image to get prediction and Grad-CAM visualization.

Choose a foot image

Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG

Browse files

diabetic_foot_ulcer.jpg 68.3KB

Uploaded Image



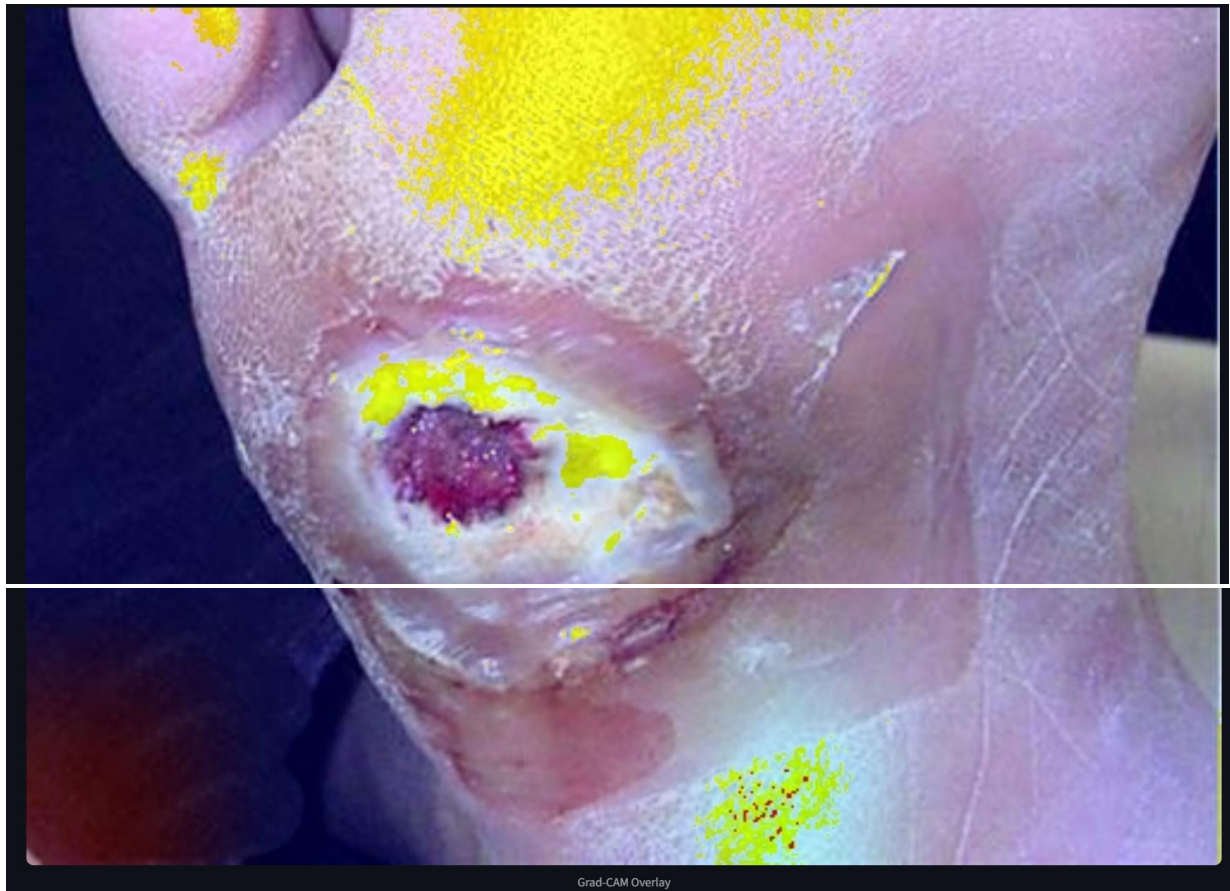
Prediction

Class: Non-Ulcer
Probability: 0.0001
Confidence: High

High confidence prediction

Grad-CAM Visualization





So, the main goals of our project is totally fulfilled.

6. Conclusion

The proposed system demonstrates that a lightweight deep learning model can be effectively used for diabetic foot ulcer detection from images. By utilizing a pretrained MobileNetV2 model with transfer learning, the system achieves a reasonable balance between performance and computational efficiency. This makes it particularly suitable for real-world deployment, especially on resource-constrained devices such as smartphones. The integration of interpretability techniques such as Grad-CAM adds significant value by providing visual explanations of the model's decision-making process, allowing users to understand which regions of the image contributed to the prediction. Additionally, the use of Monte Carlo Dropout introduces an uncertainty estimation mechanism, which enhances the reliability of the predictions by indicating the confidence level of the model's output.

Despite these strengths, the system has several limitations that affect its overall performance and generalizability. One of the primary challenges is the relatively small and heterogeneous dataset, which required extensive manual cleaning and filtering. The presence of visually similar conditions such as callus formation, pre-ulcer states, and healed ulcers introduces ambiguity, making it difficult for the model to consistently distinguish between normal and ulcer cases. Furthermore, the current binary classification approach simplifies the problem but does not fully reflect real clinical scenarios, where multiple stages and types of foot conditions exist. As a result, some borderline or early-stage cases may be misclassified, reducing the robustness of the system.

To address these limitations, several improvements can be considered in future work. Expanding the dataset with more diverse and clinically representative images would significantly enhance the model's learning capability and generalization. Transitioning from binary classification to a multi-class framework—distinguishing between normal, ulcer, callus, and other related conditions—would make the system more practical and clinically relevant. Additionally, experimenting with more advanced architectures such as EfficientNet-B0 may improve classification accuracy while maintaining efficiency. Finally, further refinement of the user interface in both the mobile and web applications can improve usability and accessibility, making the system more suitable for real-world healthcare applications.

To sum up,

Strengths:

- Lightweight model suitable for mobile deployment
- Includes explainability (Grad-CAM)
- Includes uncertainty estimation

Limitations:

- Limited dataset size
- Binary classification only
- Confusion with similar conditions (callus, healed ulcers)

Future Improvements:

- Use larger and more diverse dataset
- Multi-class classification (ulcer, callus, normal)
- Improve model accuracy with EfficientNet
- Enhance UI/UX of applications